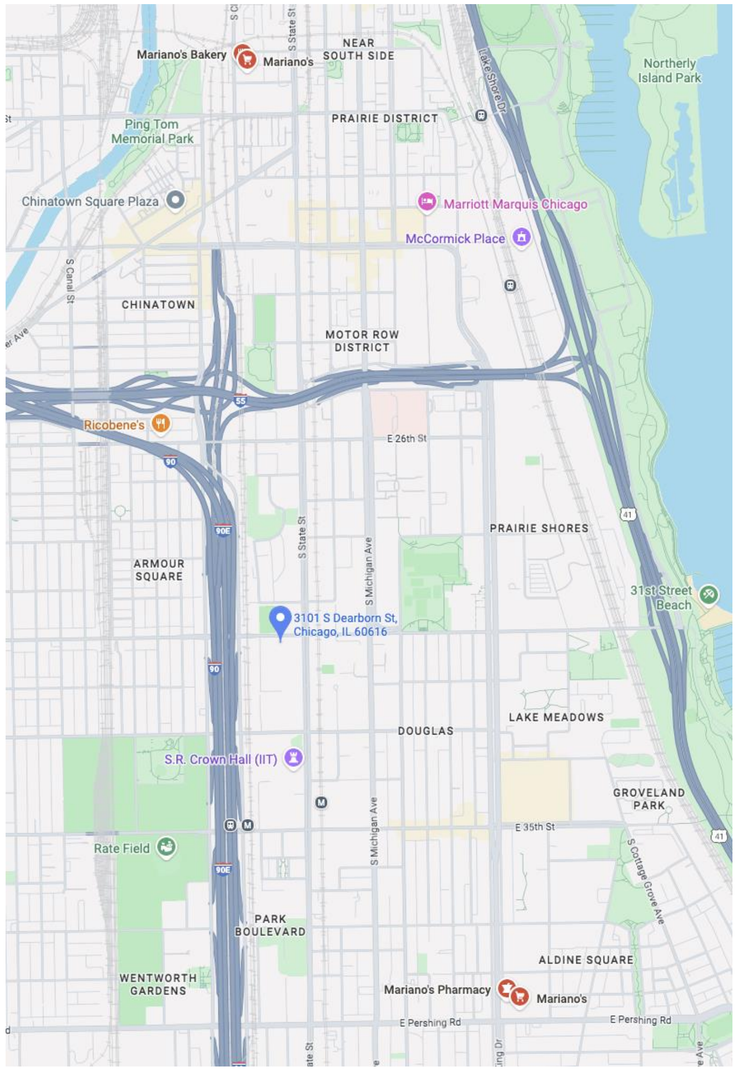
Chem 456 1/14/2026

- Start installing Juypter on SDSC Expanse (Part 1 of Exercise 1) in the Expanse Shell
- After you get started, I'll briefly describe
  - Software stacks
  - Environment variables
  - Conda
- Next you can introduce yourselves, highlighting points from the Introduce Yourself survey
- Then we'll continue with Exercise 1: Jupyter Notebooks (Part 2 and onwards) in JupyterLab

Software Stacks

- Computational science uses software packages
  - written by different people from different organizations with different styles
  - with many versions
- They don’t aways work well together

Environment Variables



If your friend said, “Let’s meet at Mariano’s for coffee and pick up some food for the party,” a good question would be, “Which one?”

Definition and Purpose

Environment variables are key-value pairs that control program behavior and system paths (which version of a program to use).

Common Variables

Common variables include $PATH to locate executables and $HOME pointing to user directories.

Usage and Commands

Commands like 'printenv' and 'echo $PATH' help users view current environment variable settings.

Modules

Module System Overview
Modules manage software versions on HPC clusters (including SDSC Expanse) by modifying environment variables.

Common Module Commands
'module avail', 'module load', 'module list', and 'module purge' are essential commands for module management.

Ensuring Reproducibility
Using modules allows specifying exact software versions in job scripts for reproducible computations.

Best Practice Workflow
Start scripts with 'module purge' and load only required modules to ensure clean environment.

Conda

Package and Environment Management
Conda manages Python packages and environments without needing administrative rights, ideal for restricted systems.

Simplified Dependency Handling
Conda avoids package conflicts by managing dependencies and creating isolated environments for projects.

Common Conda Commands
Key commands include installing packages with 'conda install' and listing installed packages with 'conda list'.

Use in HPC Environments
Conda is especially useful in high-performance computing clusters where system-wide installs are limited.

Conda Environments

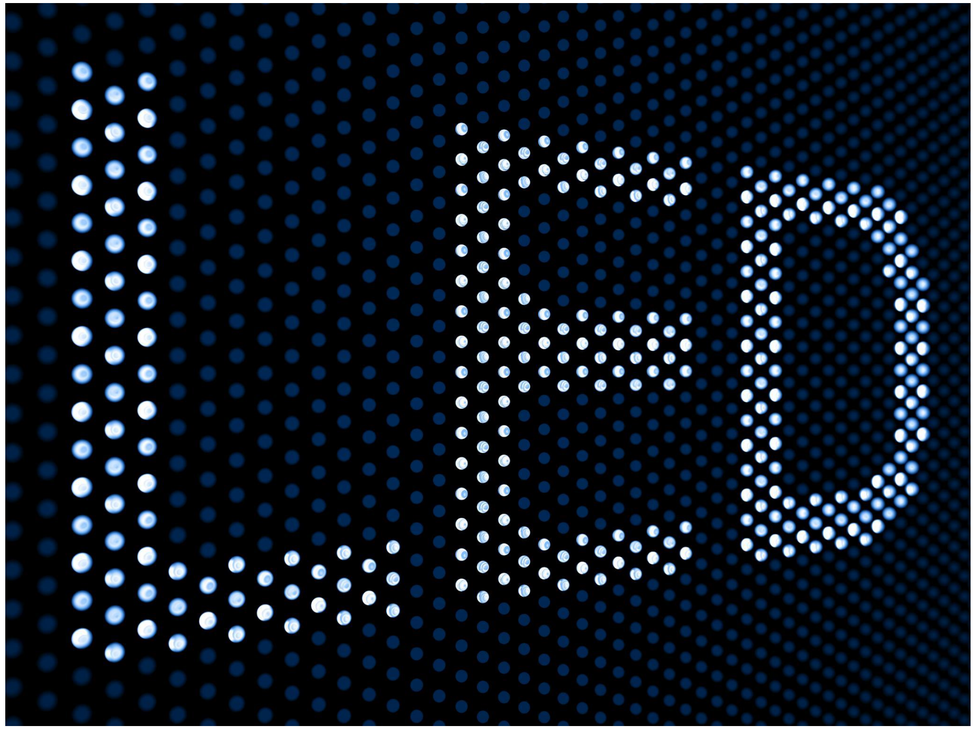
Environment Isolation
Conda environments isolate Python interpreters and packages to prevent version conflicts between projects.

Creating and Activating
Create environments with 'conda create' and activate or deactivate them using 'conda activate' and 'conda deactivate'.

Reproducibility and Export
Export environments to YAML files for reproducibility and recreate them with 'conda env create -f env.yml'.

Consistent Results
Effective environment management ensures consistent results across different systems and scientific research runs.

Reliable Workflow Sequence



Clean Environment Setup
Begin workflows with limited modules to avoid conflicts and ensure a clean starting point.

Module Loading and Conda Activation
Load required modules and activate the correct conda environment to ensure reproducibility.

Running and Organizing Scripts
Execute Python scripts and organize outputs into dedicated directories for clarity and management.

Debugging Environment Verification
Use commands like 'module list' and 'which python' to verify the active environment during debugging.

Exercise 1: Jupyter Notebooks

Homework

- By next Wednesday 1/21 at 8:30am,
  - Fill out the Introduce Yourself survey under Participation on Canvas. This counts as two participation points.
  - Complete Exercise 0.
- By next Friday 1/23 at 8:30am, complete Exercise 1.